

Assignment No: 1(A)

```
#include <iostream>
#include <vector>
#include <queue>
#include <omp.h>

using namespace std;

class Graph {
    int V;
    vector<vector<int>> adj;

public:
    Graph(int V) {
        this->V = V;
        adj.resize(V);
    }

    void addEdge(int u, int v) {
        adj[u].push_back(v);
        adj[v].push_back(u);    // undirected graph
    }

    void parallelBFS(int start) {

        vector<bool> visited(V, false);
        queue<int> q;

        visited[start] = true;
        q.push(start);

        cout << "Parallel BFS Traversal: ";

        while(!q.empty()) {

            int node = q.front();
            q.pop();

            cout << node << " ";

            #pragma omp parallel for
            for(int i = 0; i < adj[node].size(); i++) {

                int neighbor = adj[node][i];
```

```

        if(!visited[neighbor]) {
            #pragma omp critical
            {
                if(!visited[neighbor]) {
                    visited[neighbor] = true;
                    q.push(neighbor);
                }
            }
        }
    }
    cout << endl;
}
};

int main() {
    int V, E;
    cout << "Enter number of vertices: ";
    cin >> V;

    Graph g(V);

    cout << "Enter number of edges: ";
    cin >> E;

    cout << "Enter edges (u v):\n";

    for(int i = 0; i < E; i++) {
        int u, v;
        cin >> u >> v;
        g.addEdge(u, v);
    }

    int start;
    cout << "Enter starting vertex: ";
    cin >> start;

    g.parallelBFS(start);

    return 0;
}

```

Input:

Enter number of vertices: 6

Enter number of edges: 5

Enter edges (u v):

0 1

0 2

1 3

1 4

2 5

Enter starting vertex: 0

Outpsut:

Parallel BFS Traversal: 0 1 2 3 4 5